LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 5 – RELASI ANTAR KELAS

FEMBIANA DIKA SAPUTRI

103132400026

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Relasi antar kelas merupakan hubungan antara kelas-kelas dalam Pemrograman Berorientasi Objek (OOP) yang memungkinkan objek saling berinteraksi dan membentuk suatu sistem. Relasi antar kelas dapat digambarkan menggunakan diagram kelas (class diagram) yang memuat nama kelas, atribut, method, serta hubungan antar kelas.

Terdapat tiga jenis relasi utama antara lain:

1. Asosiasi (Association)

Asosiasi merupakan hubungan antara dua kelas yang berbeda, ketika objek dari kedua kelas tersebut saling berinteraksi atau menggunakan method satu sama lain. Dalam hubungan ini, objek dari suatu kelas tidak menjadi bagian atau atribut tetap dari kelas lainnya.

2. Agregasi (Aggregation)

Agregasi merupakan hubungan antara dua kelas yang menunjukkan hubungan antara keseluruhan dan bagian. Pada hubungan ini, suatu kelas menampung objek dari kelas lain sebagai bagiannya, tetapi kedua objek tetap dapat dibuat dan berdiri sendiri tanpa bergantung satu sama lain.

3. Komposisi (Composition)

Komposisi merupakan bentuk hubungan agregasi yang lebih kuat, ketika keberadaan objek bagian bergantung pada objek utama yang memilikinya. Jika objek utama dihapus, objek yang menjadi bagiannya juga ikut hilang karena siklus hidup keduanya saling bergantung.

Pemahaman terhadap ketiga relasi tersebut membantu dalam merancang struktur program yang sesuai dengan hubungan antar objek dalam suatu sistem.

B. Guided

1. guided.asosiasi

Mahasiswa.java

```
package guided.asosiasi;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama){
        this.nama = nama;
    }
    public void setNim(String nim){
```

```
        this.nim = nim;
    }
    public void setNilai(int nilai){
        this.nilai = nilai;
    }
    public String getNama(){
        return this.nama;
    }
    public String getNim(){
        return this.nim;
    }
    public int getNilai(){
        return this.nilai;
    }
}
```

Output :

-

Dosen.java

```
package guided.asosiasi;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    //Setter
    public void setKodeDosen(String kodeDosen){
        this.kodeDosen = kodeDosen;
    }
    public void setNamaDosen(String namaDosen){
        this.namaDosen = namaDosen;
    }
    //Getter
    public String getKodeDosen(){
        return this.kodeDosen;
    }
    public String getNamaDosen(){
        return this.namaDosen;
    }
    public void giveScore(Mahasiswa s, int nilai){
    // ini asosiasi, method milik class Student dipanggil di classDosen,
    // tp objek Student tidak menjadi atribut dr class Lecture
        s.setNilai(nilai);
    }
    public int getScore(Mahasiswa s){
    // ini asosiasi, method milik class Student dipanggil di class Lecture,
    // tp objek Student tidak menjadi atribut dr class Lecture
        return s.getNilai();
```

```
    }
    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :"+m.getNim());
        System.out.println("Nama :"+m.getNama());
        System.out.println("Nilai :"+d.getScore(m));
    }
}
```

Output :

```
Nim :130118383
Nama :Budi
Nilai :90
```

2. guided.agregasi

Mahasiswa.java

```
package guided.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }
    //Getter
    public String getNIM(){
        return NIM;
    }
    public String getNama(){
        return Nama;
    }
    //Setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }
    public void setNama(String Nama){
        this.Nama = Nama;
    }
}
```

Output :

-

Jurusan.java

```
package guided.agregasi;

public class Jurusan {
   private static int jumlahMahasiswa;
   private String kodeJurusan, namaJurusan;
   private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

   //Constructor
   public Jurusan (String kodeJurusan, String namaJurusan){
      this.kodeJurusan = kodeJurusan;
      this.namaJurusan = namaJurusan;
   }

   //Behaviour
   public void addMahasiswa(Mahasiswa m){
      this.daftarMahasiswa[jumlahMahasiswa] = m;
      jumlahMahasiswa++;
   }

   public void displayMahasiswa(){
      System.out.println("Kode Jurusan: " + this.kodeJurusan);
      System.out.println("Nama Jurusan: " + this.namaJurusan);

      System.out.println("Daftar Mahasiswa: ");

      for (int i = 0; i<jumlahMahasiswa; i++){
         System.out.println(i+1 + ". " + daftarMahasiswa[i].getNIM() + " " +
daftarMahasiswa[i].getNama());
      }
   }

   //Getter
   public String getKodeJurusan(){
      return kodeJurusan;
   }
   public String getNamaJurusan(){
      return namaJurusan;
   }
   //Setter
   public void setKodeJurusan(String kodeJurusan){
      this.kodeJurusan = kodeJurusan;
   }
   public void setNamaJurusan(String namaJurusan){
      this.namaJurusan = namaJurusan;
   }
}
```

Output :

-

3. guided.komposisi

Komputer.java

```
package guided.komposisi;

public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor,
String brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
}
```

Output :

-

Monitor.java

```
package guided.komposisi;

public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

Output :

-

Mouse.java

```
package guided.komposisi;

public class Mouse {
    private String brand, tipe;

    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

Output :

-

Processor.java

```
package guided.komposisi;

public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Output :

-

4. guided.Main

Main.java

```
package guided.Main;

import guided.agregasi.*;
import guided.komposisi.*;
```

```
public class Main {
    public static void main(String[] args) {
        //runGuided2();
        //runGuided3();
    }


    public static void runGuided1(){}

    public static void runGuided2(){
        Mahasiswa m1 = new Mahasiswa("103112400148", "Rendhi");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");

        Jurusan prodi = new Jurusan("SD","Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();

    }

    public static void runGuided3(){
        Komputer pc = new Komputer("Intel Core i5",3200,"LG 24
Inch","Logitech","Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output :

runGuided2();

```
Kode Jurusan: SD
Nama Jurusan: Sains Data
Daftar Mahasiswa:
1. 103112400148 Rendhi
2. 103112400149 John
```

runGuided3();

```
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
```

C. Unguided

Petunjuk Umum

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint dan tugas yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya (gunakan enkapsulasi).
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga this perlu dipakai secara eksplisit).
- Proyek wajib mengikuti konvensi penamaan standar (PascalCase untuk Class, camelCase untuk variabel/method).
- Gunakan array standar bertipe tetap (bukan ArrayList atau koleksi library lainnya).
- Tidak perlu menggunakan inheritance, overriding, maupun abstract class.

Soal - Sistem Manajemen Pesanan Cafe

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (Main).

1. Tugas 1: Implementasi Komposisi (Composition)
   Komposisi adalah relasi di mana sebuah objek memiliki objek lain sebagai bagian internalnya yang tidak dapat berdiri sendiri. Siklus hidup objek internal terikat penuh pada objek utamanya.
   Pada sistem cafe:
   - Setiap item pesanan (ItemPesanan) pasti memiliki rincian harga (RincianHarga) yang mencatat harga dasar dan pajak (10%).
   - Objek RincianHarga harus di-instansiasi langsung di dalam constructor ItemPesanan. Objek RincianHarga tidak pernah dibuat di main. Jika ItemPesanan dihapus, maka RincianHarga miliknya ikut hilang.

   Kebutuhan Tugas 1
   Buatlah dua class: RincianHarga dan ItemPesanan.
   Field yang disarankan:

Class	Field	Jenis	Tipe	Keterangan
RincianHarga	hargaDasar	Instance field	double	Harga awal sebelum pajak

RincianHarga	pajak	Instance field	double	Nilai nominal pajak (10% dari harga dasar)
ItemPesanan	namaItem	Instance field	String	Nama menu (misal: "Iced Americano")
ItemPesanan	rincian	Instance field	RincianHarga	Objek internal rincian harga (**Komposisi**)

Method yang disarankan:

Class	**Method**	**Return**	**Keterangan**
RincianHarga	getHargaTotal()	double	Mengembalikan nilai hargaDasar + pajak
ItemPesanan	getNamaItem()	String	Mengembalikan nama item pesanan
ItemPesanan	getRincian()	RincianHarga	Mengembalikan objek rincian harga

Jawaban :

Class RincianHarga.java

```
package unguided.Komposisi;

public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 0.10;
    }

    public double getHargaTotal() {
        return hargaDasar + pajak;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public double getPajak() {
        return pajak;
    }
}
```

Class ItemPesanan.java

```
package unguided.Komposisi;

public class ItemPesanan {
```

```
    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return namaItem;
    }

    public RincianHarga getRincian() {
        return rincian;
    }
}
```

Output :
-

Penjelasan :
Class RincianHarga digunakan untuk menyimpan harga dasar dan menghitung pajak sebesar 10% dari harga tersebut. Constructor menerima parameter hargaDasar, kemudian menghitung nilai pajak secara otomatis. Method getHargaTotal() mengembalikan hasil penjumlahan harga dasar dan pajak, sedangkan getHargaDasar() dan getPajak() digunakan untuk mengambil masing-masing nilai. Atribut dibuat menggunakan access modifier private agar tidak dapat diakses secara langsung dari luar kelas.

Class ItemPesanan digunakan untuk menyimpan nama menu dan rincian harganya. Pada constructor, objek RincianHarga dibuat menggunakan kata kunci new, sehingga pembentukan rincian harga dilakukan langsung di dalam class ItemPesanan.

Hubungan kedua kelas tersebut merupakan komposisi, karena objek RincianHarga menjadi bagian internal dari ItemPesanan. Method getNamaItem() digunakan untuk mengambil nama menu, sedangkan getRincian() digunakan untuk memperoleh objek rincian harga.

2. Tugas 2: Implementasi Agregasi (Aggregation)
   Agregasi adalah relasi di mana sebuah objek menampung kumpulan objek lain yang dapat berdiri sendiri. Objek yang ditampung dibuat secara independen di luar, lalu dimasukkan ke dalam objek wadah.
   Pada sistem cafe:

- Sebuah Pesanan menampung beberapa objek ItemPesanan menggunakan array standar.
- Objek ItemPesanan dibuat terpisah terlebih dahulu (di main), lalu dimasukkan satu per satu ke dalam Pesanan melalui method tambahItem(). Jika Pesanan dibatalkan atau dihapus, objek ItemPesanan tersebut secara konsep tetap ada.

Kebutuhan Tugas 2

Buatlah class Pesanan.

Field yang disarankan:

Field	**Jenis**	**Tipe**	**Keterangan**
nomorMeja	Instance field	int	Nomor meja pelanggan
daftarItem	Instance field	ItemPesanan[]	Array penampung item pesanan (**Agregasi**)
jumlahItem	Instance field	int	Penanda jumlah item yang sudah masuk ke array

Method yang disarankan:

Method	**Return**	**Keterangan**
tambahItem(ItemPesanan item)	void	Memasukkan objek ItemPesanan dari luar ke dalam array daftarItem
tampilkanPesanan()	void	Menampilkan nomor meja dan seluruh item beserta harga total per item
getNomorMeja()	int	Mengembalikan nomor meja
getDaftarItem()	ItemPesanan[]	Mengembalikan array daftar item
getJumlahItem()	int	Mengembalikan jumlah item yang tersimpan

Jawaban :

Class Pesanan.java

```
package unguided.Agregasi;

import unguided.Komposisi.*;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }
```

```
    public void tambahItem(ItemPesanan item) {
        if (jumlahItem < daftarItem.length) {
            daftarItem[jumlahItem] = item;
            jumlahItem++;
        } else {
            System.out.println("Pesanan sudah penuh.");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + nomorMeja);

        for (int i = 0; i < jumlahItem; i++) {
            ItemPesanan item = daftarItem[i];
            RincianHarga rincian = item.getRincian();

            System.out.println((i + 1) + ". "
                + item.getNamaItem()
                + " - Rp " + rincian.getHargaTotal()
                + " (Harga: " + rincian.getHargaDasar()
                + ", Pajak: " + rincian.getPajak() + ")");
        }
    }

    public int getNomorMeja() {
        return nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return daftarItem;
    }

    public int getJumlahItem() {
        return jumlahItem;
    }
}
```

Output :

-

Penjelasan :

Class Pesanan digunakan untuk menyimpan nomor meja dan daftar item yang dipesan pelanggan. Atribut daftarItem berupa array bertipe ItemPesanan[] yang berfungsi menampung beberapa objek item pesanan, sedangkan jumlahItem mencatat banyaknya item yang telah dimasukkan.

Method tambahItem() menerima objek ItemPesanan yang sudah dibuat sebelumnya, kemudian memasukkannya ke dalam array jika kapasitas masih tersedia. Method tampilkanPesanan() menampilkan nomor meja, nama menu, harga dasar, pajak, dan harga total setiap item. Sementara itu, method getter digunakan untuk mengambil nomor meja, daftar item, dan jumlah item.

Hubungan antara Pesanan dan ItemPesanan merupakan agregasi, karena objek ItemPesanan dibuat secara terpisah dan kemudian dimasukkan ke dalam Pesanan. Objek item tersebut secara konsep tetap dapat berdiri sendiri meskipun objek pesanan dihapus.

3. Tugas 3: Implementasi Asosiasi (Association)
   Asosiasi adalah relasi interaksi antara dua objek independen yang saling menggunakan tetapi tidak saling memiliki. Objek tidak menyimpan objek lain sebagai field permanen, melainkan berinteraksi melalui parameter method.
   Pada sistem cafe:
   - Seorang Kasir memproses pembayaran dari sebuah Pesanan.
   - Class Kasir tidak menyimpan objek Pesanan sebagai field, melainkan menerima objek Pesanan sebagai parameter pada method prosesPembayaran().

   Kebutuhan Tugas 3
   Buatlah class Kasir.
   Field yang disarankan:

Field	**Jenis**	**Tipe**	**Keterangan**
namaKasir	Instance field	String	Nama petugas kasir

Method yang disarankan:

Method	**Return**	**Keterangan**
prosesPembayaran(Pesanan pesanan)	void	Mengakses data Pesanan, menghitung akumulasi total bayar seluruh item, lalu mencetak struk pembayaran (**Asosiasi**)

Jawaban :
Class Kasir.java

```
package unguided.Asosiasi;

import unguided.Komposisi.*;
import unguided.Agregasi.*;

public class Kasir {
   private String namaKasir;
```

```
    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;

        ItemPesanan[] daftar = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga rincian = daftar[i].getRincian();

            totalSubtotal += rincian.getHargaDasar();
            totalPajak += rincian.getPajak();
        }

        System.out.println();
        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + namaKasir);
        System.out.println("Pesanan        : Meja "
            + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : "
            + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp "
            + (totalSubtotal + totalPajak));
        System.out.println("================================");
    }
}
```

Output :
-

Penjelasan :
Class Kasir digunakan untuk menyimpan nama kasir dan memproses pembayaran pesanan. Method prosesPembayaran() menerima objek Pesanan sebagai parameter, kemudian mengambil daftar item untuk menghitung total harga dasar dan total pajak. Perulangan for digunakan untuk menjumlahkan harga seluruh item. Setelah perhitungan selesai, program menampilkan struk yang berisi nama kasir, nomor meja, jumlah item, subtotal, total pajak, dan total pembayaran.

Hubungan antara Kasir dan Pesanan merupakan asosiasi, karena objek Pesanan digunakan melalui parameter method dan tidak disimpan sebagai atribut tetap dalam class Kasir.

4. Tugas 4: Eksekusi dan Pengujian di Main

   Buatlah class Main untuk merangkai seluruh kelas dan menguji ketiga jenis relasi.

   Alur program di main

   1) Buat 1 objek Kasir (misal: "Budi").
   2) Buat 3 objek ItemPesanan secara independen:
      a. "Iced Americano", harga dasar: 18000
      b. "Beef Croissant", harga dasar: 25000
      c. "Matcha Latte", harga dasar: 22000
   3) Buat 1 objek Pesanan untuk Meja 4 (kapasitas array misal: 5 item).
   4) Masukkan ketiga objek ItemPesanan ke dalam objek Pesanan menggunakan method tambahItem().
   5) Tampilkan rincian pesanan dengan memanggil tampilkanPesanan() pada objek Pesanan.
   6) Jalankan pemrosesan pembayaran dengan memanggil method prosesPembayaran(pesanan) dari objek Kasir.

   Jawaban :

   Class Main.java

```
package unguided.Main;

import unguided.Komposisi.*;
import unguided.Agregasi.*;
import unguided.Asosiasi.*;

public class Main {
    public static void main(String[] args) {
        Kasir kasir = new Kasir("Budi");

        ItemPesanan item1 =
            new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 =
            new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 =
            new ItemPesanan("Matcha Latte", 22000);

        Pesanan pesanan = new Pesanan(4, 5);

        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        pesanan.tampilkanPesanan();
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

Penjelasan :

Class Main merupakan kelas utama untuk menjalankan program dan menguji ketiga jenis relasi antar kelas. Tahapan yang dilakukan adalah sebagai berikut.

1) Membuat objek Kasir dengan nama Budi.
2) Membuat tiga objek ItemPesanan secara terpisah, yaitu Iced Americano seharga Rp18.000, Beef Croissant seharga Rp25.000, dan Matcha Latte seharga Rp22.000.
3) Membuat objek Pesanan untuk meja 4 dengan kapasitas lima item.
4) Memasukkan ketiga objek item ke dalam pesanan menggunakan method tambahItem().
5) Menampilkan daftar pesanan melalui method tampilkanPesanan().
6) Memanggil method prosesPembayaran() untuk menghitung total pembayaran dan menampilkan struk.

Urutan tersebut menunjukkan bagaimana seluruh kelas saling berhubungan untuk menjalankan sistem manajemen pesanan cafe.

D. Kesimpulan

Praktikum ini memberikan pemahaman mengenai konsep relasi antar kelas dalam Pemrograman Berorientasi Objek menggunakan Java. Asosiasi, agregasi, dan komposisi memiliki karakteristik yang berbeda berdasarkan bentuk hubungan serta ketergantungan antar objek. Pemahaman terhadap perbedaan tersebut penting untuk menentukan struktur

kelas yang tepat sesuai kebutuhan program. Selain itu, penerapan konsep OOP yang baik dapat membantu menghasilkan program yang terstruktur, terorganisasi, dan lebih mudah dipahami serta dikembangkan.